



Birzeit University

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

ENCS4320 – Applied Cryptography (Term 1252)

Final Course Project: SecureChannel — A Secure Communication Application

1. Overview

In this project, you will design and implement, **from scratch**, a complete secure communication system in Python. Two parties (e.g., a client and a server, or two peers) will establish a shared secret over an insecure network, authenticate each other, and then exchange messages with **confidentiality, integrity, and authenticity** guarantees, including support for **associated data**.

The project ties together the main themes of the course: symmetric-key cryptography, asymmetric-key cryptography, cryptographic hash functions, message authentication, authenticated encryption with associated data (AEAD), and secure key establishment.

You are required to implement **modern, currently deployed algorithms** — the same primitives used today in TLS 1.3, SSH, Signal, and WireGuard. Each primitive is fully specified in a short, self-contained standards document (RFC/FIPS), and Section 5 provides a curated set of references for every component. Part of this project is learning to implement a cryptographic algorithm directly from its official specification — a core skill of the working security engineer.

2. Learning Objectives

By completing this project, you will be able to:

1. Implement modern cryptographic primitives from their specifications (RFCs) and validate them against official test vectors.
2. Compose primitives correctly into a secure protocol (key exchange → key derivation → authenticated encryption), and understand why each step is necessary.
3. Reason about practical attacks: replay, reflection, nonce reuse, tampering, man-in-the-middle.
4. Defend your design decisions and explain your code in a technical discussion.

3. System Description

Your application consists of two programs (or one program with two roles) communicating over TCP sockets on localhost or a LAN. The session proceeds in two phases:

Phase A — Handshake (Key Establishment & Authentication)

1. **Ephemeral key exchange:** Each party generates an ephemeral **X25519** key pair and they perform an Elliptic-Curve Diffie–Hellman exchange (RFC 7748) to derive a shared secret.
2. **Authentication:** The two parties share a long-term **pre-shared key (PSK)** established out-of-band (e.g., read from a local configuration file — think of it as a strong password the two users agreed on in person). Each party authenticates the handshake by computing **HMAC(PSK, transcript)** over the full handshake transcript (both ephemeral public keys + both identities + protocol version) and sending it to the other side for verification. A mismatching tag must abort the connection. This binds the exchanged ephemeral keys to the two legitimate parties and defeats man-in-the-middle attacks.

Bonus alternative: replace (or supplement) PSK authentication with **Ed25519 digital signatures** over the transcript — see Section 9.

3. **Key derivation:** The shared secret is fed into **HKDF** (RFC 5869, instantiated with your own HMAC-SHA-256, with the PSK mixed in as the salt) to derive two independent directional session keys (client→server and server→client), plus the initial nonce material.

Phase B — Secure Messaging (AEAD Channel)

4. **Authenticated encryption:** All application messages are protected with **ChaCha20-Poly1305** (RFC 8439).
5. **Associated data:** Each message carries a plaintext header (e.g., protocol version, message type, sender ID, sequence number). The header is **not encrypted** but **must be authenticated** as associated data — any tampering with the header must cause decryption to fail.
6. **Replay protection:** Use strictly increasing sequence numbers (bound to the nonce and/or the associated data) so that replayed or reordered ciphertexts are detected and rejected.
7. **User interface:** A simple command-line chat (interactive send/receive) is sufficient. A file-transfer mode is a welcome addition but not required.

4. Implementation Rules — "From Scratch"

This is the heart of the project. The following must be implemented **by you, in pure Python**, from the official specifications:

Component	Specification	Underlying concept
SHA-256	FIPS 180-4	Secure hash functions
HMAC	RFC 2104	Message authentication codes
HKDF (extract + expand)	RFC 5869	Key derivation from a shared secret
ChaCha20 stream cipher	RFC 8439	Symmetric encryption / stream ciphers
Poly1305 MAC	RFC 8439	One-time MACs
ChaCha20-Poly1305 AEAD	RFC 8439 §2.8	Authenticated encryption with associated data
X25519 key exchange	RFC 7748	Diffie–Hellman over elliptic curves

You are not allowed to use cryptographic libraries **inside your implementation** of the above-mentioned components. However, you *may* use these libraries **in your test files**, to cross-check your output against a reference implementation — and we encourage you to do so.

Mandatory test vectors: Your repository must include a test suite that validates every primitive against the official test vectors published in its RFC/FIPS document. A primitive that does not pass the official test vectors is considered not implemented.

5. References

Every primitive in this project is fully specified in a freely available, self-contained standards document. These specifications — together with the supporting references below — contain everything you need to read, understand, and implement each component. Reading a specification carefully *is* part of the project.

Component	Primary specification (implement from this)	Supporting reading
SHA-256	NIST FIPS 180-4, <i>Secure Hash Standard</i> , §5–6 (includes pseudocode and constants)	Paar, Pelzl & Güneysu, <i>Understanding Cryptography</i> , 2nd ed., Ch. 11 (Hash Functions)
HMAC	RFC 2104 (5 pages); also NIST FIPS 198-1	Paar, Pelzl & Güneysu, Ch. 13 (MACs)
HKDF	RFC 5869 (10 pages, includes test vectors)	Krawczyk, <i>Cryptographic Extraction and Key Derivation: The HKDF Scheme</i> (the design paper, optional)
ChaCha20, Poly1305, and the AEAD construction	RFC 8439 — exceptionally readable; walks through every computation with intermediate values and provides complete test vectors	Paar, Pelzl & Güneysu, Ch. 2 (Stream Ciphers — includes Salsa20/ChaCha20); Boneh & Shoup, <i>A Graduate Course in Applied Cryptography</i> (free online), Ch. 9 (AEAD)
X25519	RFC 7748, §4–5 (includes the Montgomery-ladder pseudocode and test vectors)	M. Kleppmann, <i>Implementing Curve25519/X25519: A Tutorial on Elliptic Curve Cryptography</i> — an excellent step-by-step implementation guide written for students; Paar, Pelzl & Güneysu, Ch. 9 (Elliptic Curve Cryptosystems)
Ed25519 (bonus only)	RFC 8032 (includes SHA-512 dependency and test vectors)	The Kleppmann tutorial above also covers Ed25519
Protocol design (handshake, transcripts, replay protection)	—	The Noise Protocol Framework specification (noiseprotocol.org) and RFC 8446 (TLS 1.3), §2 — for inspiration on how real protocols structure handshakes; you are not required to follow them

Suggested reading order: start with RFC 8439 — it is self-contained and will give you your first passing test vectors within days — then FIPS 180-4 and RFC 2104/5869, and tackle RFC 7748 last, with the Kleppmann tutorial alongside it.

6. Teams and GitHub Repository Requirement

The project is done in **teams of two students**. Register your team using the link below; fill in both students' **names and IDs**, together with a **link to your private GitHub repository**.

Team Registration Form:

<https://docs.google.com/spreadsheets/d/1Jep9VWaMLH9nSsMIXpNxjyEHujzKOXD>

All teams must maintain a **private GitHub repository** for their project throughout the project period. Requirements:

- Create a private GitHub repository as soon as your team is formed.
- Share the repository link with the instructors (via the sign-up sheet) **no later than the registration deadline**.
- **Commit and push your work regularly throughout the project period — do not make one large commit at the end.** Use meaningful commit messages, and commit from both team members' accounts.

Important: Commit history will be reviewed. A repository with a single large commit the night before the deadline is a red flag: it indicates the work was not carried out as required, will result in a deduction from the code-quality component, and may be escalated as an academic-integrity case. Sustained, incremental commits demonstrate genuine ongoing effort.

At a minimum, the repository should contain: the main source code (updated regularly), a `README.md` with the project title, team members, a brief description, and instructions for running the two parties and the test suite, the test files with the official test vectors, a `requirements.txt` listing any dependencies used in testing, and a short report describing your protocol design: message formats, key schedule, nonce construction, and how you prevent replay and tampering. **Do not commit long-term private keys, pre-shared keys, or other secret material** — generate them locally and document the procedure in the `README`.

7. Ethical Use of AI Tools

AI-assisted tools (e.g., GitHub Copilot, ChatGPT, Claude, Gemini, and similar) are permitted in this course project subject to the following conditions. Misuse is treated as academic dishonesty.

Permitted Uses:

- Debugging code, understanding error messages, or looking up syntax.
- Getting feedback on the clarity or grammar of your report and `README`.
- Exploring ideas or understanding concepts — e.g., asking how the Montgomery ladder works, or why Poly1305 keys must be one-time (treat this like consulting a tutor).
- Generating boilerplate or utility code (e.g., socket handling, command-line parsing, test harness scaffolding).

Not Permitted:

- Having an AI tool write the core cryptographic primitives or the protocol logic (handshake, key schedule, AEAD channel) on your behalf.
- Submitting AI-generated text as your own design report, analysis, or conclusions without meaningful independent editing and verification.
- Using AI tools to generate explanations you have not independently understood.

Mandatory AI Usage Statement: Every team must include an **AI Usage Statement** as a dedicated section in their report. This statement must describe: (1) which AI tools were used, if any; (2) specifically how they were used; and (3) which parts were written or created entirely by the team without AI assistance. Teams that did not use any AI tools must state this explicitly. Omission of this statement when AI tools were used is treated as a violation of academic integrity.

Guiding Principle: You are responsible for every claim, result, and line of code in your submission. The oral discussion is specifically designed to assess genuine understanding — be prepared to explain, defend, and modify every aspect of your project. Code you cannot explain is not your code, and will be graded accordingly — regardless of how well it runs.

8. Oral Discussion

A substantial portion of the grade comes from an **individual oral discussion**, held during the final-exam period, in which each team member will be asked — separately — to explain and modify *any* part of the submitted code. Typical questions include: *What happens if a nonce is reused, and show me how your code guarantees it never happens. Walk me through your HKDF expand step — how are the two directional session keys derived? Why must the handshake HMAC cover both ephemeral public keys — what attack succeeds if it covers only one? Show me where your code verifies the Poly1305 tag, and demonstrate what happens when an attacker modifies one byte of the header. How does your receiver detect a replayed message? Change the code so the AAD also includes a timestamp — live.*

Discussion questions focus on **your implementation and the material in the references of Section 5** — what your code does, why each step of the specification is followed, and what breaks if it is not. You will not be examined on theoretical design rationale beyond that material (e.g., why the designers of Curve25519 chose its specific parameters).

A discussion **time-slot sign-up sheet** will be distributed so each team can choose the slot most convenient for them within the final-exam period. Both team members must attend; each is questioned and graded individually.

9. Grading

Component	Weight
Correctness of primitives (official test vectors pass)	25%
Protocol implementation (handshake, AEAD channel, AAD, replay protection)	25%
Code quality, repository hygiene, commit history, README and report	10%
Individual oral discussion (understanding, ability to explain and modify the code)	40%

Bonus (up to 10%): any of the following — only considered if the core requirements are complete and the discussion demonstrates genuine understanding:

- **Ed25519 digital signatures** (RFC 8032, including your own SHA-512) replacing or supplementing PSK authentication of the handshake.
- Post-quantum hybrid key exchange (e.g., combining X25519 with ML-KEM-512 from FIPS 203).
- A file-transfer mode with chunked AEAD, or a GUI.

10. Academic Integrity & Collaboration Policy

Projects are team efforts. Within your team, full collaboration is expected and encouraged. Across teams, the following applies:

- It is acceptable to consult existing resources — RFCs, textbooks, pseudocode, and educational references found online — but you must **document every such source clearly** in your report. Remember that the from-scratch rule in Section 4 still applies: consulting a reference is allowed; copying an implementation is not.
- You will be graded on your **value-add** — what your team understood, built, and analyzed beyond what any external resource provided.
- Each team member must include a brief **individual contribution statement** in the report describing their specific role and contributions.
- In almost all cases every team member receives the same project score. However, the instructors reserve the right to differentiate scores in egregious cases where contribution is clearly imbalanced — including through the individual oral discussion.

The following are **violations of academic integrity** and will result in a grade of zero, with the possibility of further disciplinary action per university policy:

- Plagiarism of text, code, or results from any source — published papers, online repositories, other students' work, or AI-generated content submitted without attribution.
- Sharing project materials with other teams or using another team's materials.
- Fabricating or manipulating test results, or claiming test vectors pass when they do not.
- Fabricating a commit history.

When in doubt, ask the instructors.

11. Deadlines

Milestone	Date
Team & repository registration deadline	14 June 2026
Repository must show meaningful progress (primitives + test vectors)	20 June 2026
Final commit deadline (repository frozen)	25 June 2026
Oral discussions	Final-exam period — slot sign-up sheet to be distributed

Late submissions will not be accepted unless prior written approval has been granted by the instructor.

By the end of this project, you will have built — line by line, from the official specifications — the same cryptographic machinery that secures every TLS connection you open and every message you send. Very few engineers can claim that.

Good luck — and have fun!